

DOUBLE S.E AUTOS

FRONTEND TECHNICAL ARCHITECTURE & DEVELOPMENT STANDARDS

Technology baseline: React + TypeScript. **Objective:** build a scalable, maintainable, high-performance automotive web application with strict separation of concerns, reusable UI, predictable state management, and a clean path from mock data to production APIs.

Document purpose. This document defines how the frontend should be structured, how responsibilities are separated, where data and logic belong, and the code-quality rules every implementation should follow.

Core principle: each file, component, hook, service, context, route, and folder should have one clear reason to change.

Architecture target: production-ready, responsive, accessible, testable, API-ready, and capable of growing into inventory management, enquiries, bookings, authentication, saved vehicles, comparison, financing, and CMS/admin integrations.

01

Engineering Principles

The codebase should favor clarity over cleverness. Architecture exists to make change safer and easier, not to create unnecessary abstraction.

- **Separation of concerns:** UI, business logic, data access, routing, state, and static configuration must remain distinct.
- **Single responsibility:** a component or module should solve one well-defined problem.
- **Composition over large components:** complex screens should be assembled from focused components and hooks.
- **DRY with judgment:** extract genuinely repeated behavior; do not create abstractions for code used once merely to appear reusable.
- **KISS:** choose the simplest architecture that satisfies current requirements while leaving clean extension points.
- **SOLID thinking:** especially single responsibility, dependency direction, and interface-driven boundaries.
- **Explicit data flow:** make it clear where data originates, how it is transformed, and which layer owns mutations.
- **Strict TypeScript:** types are part of the architecture, not decoration.

02

Recommended Source Architecture

The exact names can evolve, but responsibilities should remain stable. Avoid placing unrelated code in generic folders simply because it is convenient.

```
src/
  assets/
    images/
    icons/
    fonts/
  components/
    common/
    layout/
    vehicle/
    motion/
  pages/
    Home/
    Inventory/
    VehicleDetails/
    About/
    Contact/
    NotFound/
  services/
    api/
    vehicleService.ts
    enquiryService.ts
  interfaces/
    vehicle.interface.ts
    enquiry.interface.ts
    api.interface.ts
  hooks/
    useVehicleFilters.ts
    useDebounce.ts
    useMediaQuery.ts
  context/
    AppContext.tsx
    InventoryContext.tsx
  routes/
    AppRoutes.tsx
    routePaths.ts
  data/
    vehicles.mock.ts
    navigation.data.ts
  constants/
    app.constants.ts
    vehicle.constants.ts
  utils/
    currency.ts
    formatters.ts
    validators.ts
  config/
    env.ts
  styles/
    globals.css
    tokens.css
  App.tsx
  main.tsx
```

Folder Responsibilities

Folder	Responsibility
components	Reusable presentation and domain UI. No page-level routing responsibility.
pages	Route-level composition. Pages orchestrate components; they should not become giant UI files.
services	External data access: API requests, transport mapping, and service-level errors.
interfaces	Domain contracts and TypeScript shapes shared across modules.
hooks	Reusable stateful behavior and React-specific logic.
context	Shared client-side application state that must be consumed across distant branches.

Folder	Responsibility
routes	Route declarations, path constants, route guards/layout routing.
data	Mock/static content used by the UI; never mix mock data into components.
constants	Stable application constants, enums/lookup values, and non-changing configuration.
utils	Pure helpers with no React rendering responsibility.
config	Environment-aware application configuration.
assets	Images, icons, fonts, and other bundled static media.

03

Component Architecture

Reusable components belong in **components**. Route-level components belong in **pages**. A page may coordinate data and compose multiple components, but repeated visual structures must be extracted.

- Keep components small enough that their purpose is obvious from the file name.
- Do not mix API calls directly into presentational components.
- Do not define large blocks of static vehicle data inside JSX.
- Do not duplicate vehicle-card, button, badge, modal, gallery, filter, or specification markup across pages.
- Prefer composition and children for flexible UI rather than giant components controlled by many boolean props.
- A component should expose a small, intentional public API.
- Do not create a universal component with dozens of unrelated modes.

Rule: if a component handles fetching, filtering, modal state, analytics, animation orchestration, formatting, and rendering at the same time, it has too many responsibilities.

04

State Management and Context

Context should store **shared application state**, not every piece of shared data. Use the narrowest state scope that solves the problem.

- **Local state:** keep state inside the component when only that component or its immediate children need it.
- **Custom hook:** extract reusable stateful behavior when multiple components need the same logic.
- **Context:** use for state required by distant parts of the tree, such as selected/saved vehicles, global filters, authentication, theme, or cross-page UI state.
- **Static data:** keep in data/constants, not Context.
- **Server data:** obtain through the service/data-access layer. Do not treat Context as a substitute for a server-state strategy.

Avoid Prop Drilling

Do not pass data through multiple intermediate components that do not use it. If state is truly cross-cutting, expose it through a focused Context and a custom consumer hook.

```
// Preferred consumer API
const { savedVehicles, toggleSavedVehicle } = useInventoryContext();

// Avoid chains such as:
App -> Layout -> Page -> Section -> Card -> Button
// when Layout, Page and Section only forward the same props.
```

Do not use Context merely to avoid passing a prop one level. Excessive Context creates hidden dependencies and unnecessary re-renders. The goal is **intentional state ownership**, not zero props.

05

Service Layer and API Boundary

The service layer is the frontend's boundary to external systems. Components and pages should request domain operations rather than construct raw network requests.

- Centralize HTTP client configuration, base URLs, headers, timeouts, and common response handling.
- Create domain-oriented services such as `vehicleService` and `enquiryService`.
- Keep endpoint strings and request mapping out of components.
- Convert transport/API responses into stable frontend domain shapes where necessary.
- Throw or return predictable errors that the UI can handle.
- Never expose secrets in client-side environment variables.
- Design mock data so it can be replaced by service calls without rewriting presentation components.

```
// UI / hook
const vehicles = await vehicleService.getVehicles(filters);

// Service responsibility
getVehicles(filters) -> request -> validate/map response -> return Vehicle[]

// Component responsibility
render loading / empty / error / success states
```

06

TypeScript Standards

- Enable strict TypeScript compiler settings.
- Avoid **any**. Use unknown when a value is genuinely unknown, then narrow it safely.
- Type component props, service inputs/outputs, Context values, hooks, events, and domain models.
- Use unions or enums for constrained domain values such as transmission, fuel type, and availability status.
- Prefer meaningful domain names such as `Vehicle`, `VehicleFilter`, `EnquiryPayload`, and `ApiResponse`.
- Do not duplicate identical interfaces in multiple folders.
- Use optional properties only when the domain truly permits absence.
- Do not silence compiler errors with unsafe assertions unless the invariant is proven and documented.

07

Hooks and Reusable Logic

Custom hooks should encapsulate reusable React behavior, not become miscellaneous utility containers.

- Use hooks for reusable state/effect logic such as filtering, debouncing, media queries, saved vehicles, or URL query synchronization.

- Hooks should have clear inputs and predictable return values.
- Keep pure formatting/calculation logic in utils, not hooks.
- Do not hide unrelated side effects inside generic hooks.
- Respect React hook rules and keep effect dependency behavior explicit.
- Avoid unnecessary effects; derive values during render or with memoization only when appropriate.

08

Routing

- Keep route definitions centralized in routes.
- Use path constants rather than repeating route strings throughout the application.
- Use route-level lazy loading where it meaningfully reduces initial bundle size.
- Provide a dedicated Not Found page.
- If authentication is introduced later, isolate route-protection logic instead of embedding authorization checks in every page.
- Vehicle detail routes should use stable identifiers/slugs suitable for direct linking and SEO.

09

Data and Domain Modeling

Vehicle information should be treated as domain data rather than arbitrary card content.

- Define one canonical Vehicle model used across inventory, detail pages, search, saved vehicles, and future API integration.
- Separate raw API models from frontend domain models if backend response shapes are unstable or backend-specific.
- Keep mock vehicle records in data files during development.
- Do not hard-code price formatting, mileage formatting, or status wording repeatedly in JSX; centralize formatting rules.
- Use stable unique IDs for rendering and routing; never use array indexes as identity for dynamic vehicle collections.

10

Forms and Validation

- Use controlled, typed form state or a dedicated form solution when complexity warrants it.
- Keep validation rules centralized and reusable.
- Validate both user experience requirements and server constraints.
- Show field-level errors clearly and accessibly.
- Prevent duplicate submissions while a request is in progress.
- Model loading, success, and failure states explicitly.
- Never rely on client-side validation for security; backend validation remains authoritative.

11

Loading, Empty, Error and Success States

Every data-driven feature must be designed for more than the happy path.

- Loading: use appropriate skeletons/spinners without causing layout shift.
- Empty: explain what is missing and provide a useful next action.

- Error: present a human-readable message and a retry path where appropriate.
- Success: confirm completed enquiries, saved actions, or submissions clearly.
- Do not leave rejected promises or failed images to silently break the interface.

12

Responsive and Accessible UI

- Build mobile-first and test across mobile, tablet, laptop, desktop, and wide displays.
- Use semantic HTML before adding ARIA.
- All interactive elements must be keyboard accessible and have visible focus states.
- Provide meaningful alt text for vehicle photography; decorative imagery should not create noise for assistive technology.
- Maintain sufficient contrast and readable text sizes.
- Respect reduced-motion preferences for non-essential animations.
- Do not make hover the only way to discover essential information.
- Ensure galleries, dialogs, menus, and filters have accessible interaction patterns.

13

Automotive Media and Motion Performance

The website depends heavily on high-resolution vehicle imagery and motion, so media must be treated as a performance concern.

- Use responsive image sources and modern formats where supported.
- Do not ship original full-resolution photography when a smaller display asset is sufficient.
- Lazy-load below-the-fold media.
- Preload only critical hero assets.
- Reserve media dimensions to prevent cumulative layout shift.
- Prefer transform and opacity for frequent animations.
- Avoid excessive scroll listeners and layout-triggering animation.
- Use motion to support product storytelling, not to obstruct browsing.
- Provide static/reduced-motion alternatives for expensive or non-essential effects.

14

Performance Standards

- Keep the initial bundle lean through route-level splitting and intentional imports.
- Avoid importing entire utility or icon libraries when only a small subset is required.
- Memoize only when profiling or component behavior justifies it; do not blanket the application with memo/useMemo/useCallback.
- Avoid unnecessary Context re-renders by separating unrelated global concerns.
- Prevent repeated network requests where caching or shared request state is appropriate.
- Monitor Core Web Vitals, especially LCP, CLS, and INP.
- Treat the hero vehicle image/video as a critical performance asset.

15

SEO and Discoverability

- Use semantic heading hierarchy and meaningful page titles.
- Provide route-specific metadata for inventory and vehicle detail pages.
- Use descriptive URLs/slugs.
- Add structured data where appropriate for dealership/vehicle content once production data is available.
- Ensure important textual content is not only embedded inside images or animation.
- Create crawlable navigation and meaningful internal links.
- Plan for sitemap and robots configuration in deployment.

16

Security and Configuration

- Keep environment-specific configuration in a typed config layer.
- Only variables intended for the browser may be exposed to the frontend.
- Never commit API secrets, private keys, credentials, or privileged tokens.
- Treat user-provided content as untrusted.
- Avoid unsafe HTML rendering. If unavoidable, sanitize content before rendering.
- Use HTTPS production endpoints.
- Do not store sensitive authentication data casually in localStorage; choose an authentication strategy based on backend security requirements.
- Keep dependency versions maintained and review security advisories.

17

Naming and File Conventions

- Use descriptive names that communicate responsibility.
- React components: PascalCase. Hooks: useXxx. Utility/service functions: camelCase.
- Avoid vague names such as helper, stuff, data2, temp, commonFunction, or component1.
- Avoid custom function names that can be confused with built-in APIs or HTTP methods; prefer explicit names such as fetchVehicles, requestVehicleDetails, or submitEnquiry.
- Keep naming consistent between file, exported component, interface, and domain terminology.
- Use barrel/index exports only when they improve clarity; avoid deep circular dependency chains.

18

Import and Dependency Rules

Dependencies should point inward toward stable contracts and shared primitives, not create circular relationships.

- Pages may import components, hooks, services, interfaces, constants, and utilities.
- Reusable components should not import route-level pages.
- Services should not import UI components.
- Interfaces/domain types should remain independent of rendering.
- Utilities should remain pure and should not depend on React unless explicitly React-specific.
- Context may use services/hooks where justified, but avoid making every feature depend on one global Context.

- Use path aliases if configured consistently and supported by TypeScript/build tooling.

19

Testing Strategy

- Unit-test pure business rules, formatters, validators, and complex utilities.
- Component-test important interactive behavior and accessibility.
- Integration-test critical flows such as inventory filtering, vehicle enquiry, and saved vehicles.
- Use end-to-end tests for the highest-value customer journeys when the application reaches production maturity.
- Test behavior rather than implementation details.
- Mock network boundaries rather than rewriting internal component behavior for tests.

20

AI-Generated Code Rules

AI may accelerate implementation, but it must follow the architecture. Generated code is not automatically production code.

- Do not accept giant single-file pages generated for convenience.
- Do not allow AI to duplicate existing components, interfaces, constants, or service logic.
- Before creating a new abstraction, check whether an existing module already owns that responsibility.
- Generated TypeScript must compile without suppressing errors through any.
- Generated UI must include responsive and accessible behavior.
- Generated network logic must go through the service layer.
- Generated shared state must follow the local state -> hook -> Context decision hierarchy.
- Review imports, dependency direction, error states, and cleanup of effects before merging.
- Do not allow placeholder mock data to leak into production components.

21

Git and Code Review Standards

- Keep commits focused and descriptive.
- Do not combine unrelated refactors and features in one change unless necessary.
- Run type checks, linting, tests, and production build checks before merging.
- Review architecture as well as visual output.
- A pull request should explain what changed, why, how it was tested, and any architectural trade-offs.
- Remove dead code, console debugging, unused imports, and temporary assets before merge.

22

Scalability Path

The architecture should support future growth without forcing today's frontend to implement tomorrow's complexity.

- Replace mock vehicle data with a backend/CMS through services.
- Add server-state caching when live inventory volume and mutation patterns justify it.
- Introduce authentication through isolated auth services/context and protected routes.

- Add saved vehicles and comparisons as focused domain features.
- Integrate enquiry/booking APIs without coupling forms to transport details.
- Support an admin/CMS separately rather than mixing dealership-management concerns into customer-facing components.
- Add analytics through a dedicated adapter/service instead of scattering vendor calls throughout components.

23

Definition of Done

A feature is not complete because it looks correct on one screen. Before considering work finished, verify:

- TypeScript compiles cleanly.
- No unnecessary any types or suppressed errors.
- Responsibilities are placed in the correct layer.
- No avoidable prop drilling or unnecessary global Context.
- No duplicated reusable UI or business logic.
- Loading, empty, error, and success states are handled.
- Responsive behavior is verified.
- Keyboard and accessibility basics are verified.
- Images and motion are optimized.
- Routes and direct links work.
- No secrets or environment-specific hard-coding exist.
- Linting/tests/build checks pass.
- The implementation can be understood by another developer without reverse-engineering a giant component.

24

Architecture Decision Guide

Question	Preferred location
Is it route-level screen composition?	pages
Is it reusable visual UI?	components
Is it reusable React state/effect behavior?	hooks
Is it shared cross-tree client state?	context
Is it API/network access?	services
Is it a domain/type contract?	interfaces
Is it static/mock content?	data
Is it a stable constant?	constants
Is it a pure transformation/formatter?	utils
Is it environment-specific setup?	config

Final rule: architecture should make the next change easier. If a solution makes today's feature work but forces unrelated modules to know about each other, reconsider the boundary.